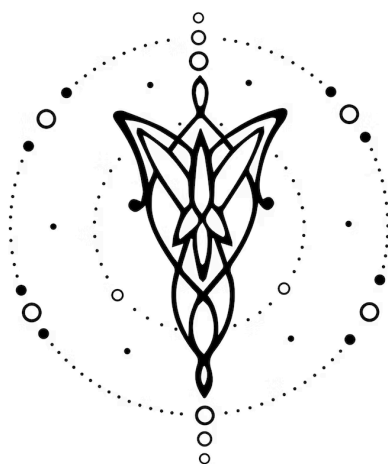




Informe Final

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE MUSIC DSL COMPILER

REPOSITORIO: [HTTPS://GITHUB.COM/NMAZZITELLI/MUSIC-DSL-COMPILER.GIT](https://github.com/nmazzitelli/music-dsl-compiler.git)

EQUIPO

Nombre	Apellido	Legajo	E-mail
Nicolás	Mazzitelli	62334	nmazzitelli@itba.edu.ar
Agustín	Brunero	62013	abrunero@itba.edu.ar
Mateo	Pirola	62810	mpirolapaulovich@itba.edu.ar

Indice

1. Introduccion	3
2. Modelo Computacional	3
2.1 Dominio	3
2.2 Lenguaje	3
3. Implementacion	4
3.1 Frontend	4
3.2 Backend	5
3.3 Adicionales	6
3.4 Dificultades Encontradas	6
4. Futuras Extensiones	6
5. Conclusiones	7
6. Referencias	7
7. Bibliografia	7

1. Introduccion

Music DSL Compiler es un compilador para un lenguaje específico de dominio orientado a la escritura de piezas musicales simples y su generación como archivos MIDI estándar. El objetivo del proyecto es permitir que una composición se exprese en texto mediante configuraciones globales, pistas, instrumentos, notas, acordes, silencios, repeticiones, variables y condicionales, manteniendo una sintaxis compacta y legible.

El proyecto cubre todos los puntos pedidos, análisis léxico, análisis sintáctico, construcción de AST, análisis semántico, generación de un modelo musical intermedio y escritura de un archivo MIDI reproducible. El repositorio incluye los scripts de compilación y prueba, ejemplos de uso y tests de aceptación y rechazo.

El compilador se ejecuta dentro de Docker. El script `src/main/bash/run.sh` recibe un archivo `.music`, invoca el compilador y deja el resultado en `out/<nombre-del-programa>.midi`.

2. Modelo Computacional

2.1 Dominio

El dominio modelado es el de una composición musical secuenciada. La unidad superior es una composición con configuraciones globales y una o más pistas. Cada pista posee un nombre, un instrumento y una secuencia de eventos. Los eventos posibles son notas, acordes, silencios, declaraciones de variables, repeticiones y condicionales.

Las configuraciones globales actualmente soportadas son:

- tempo, expresado en BPM.
- time signature, expresado como numerador y denominador.
- key, expresado como nota base y modo mayor o menor.

La generación toma este modelo y lo traduce a MIDI. Cada pista del lenguaje se transforma en una pista MIDI con canal asignado. Los instrumentos melódicos usan canales 0-8 y 10-15, mientras que drums usa el canal 9, que es el canal reservado por convención MIDI para percusión.

El tiempo se representa con 480 ticks por negra. Las duraciones del lenguaje se mapean de forma fija: whole a 1920 ticks, half a 960, quarter a 480, eighth a 240 y sixteenth a 120.

2.2 Lenguaje

Un programa comienza con configuraciones globales y continúa con una o más pistas:

```
tempo 120
time signature 4/4
key C major

track melody instrument piano {
  integer loud = 100
  play C4 quarter velocity loud
  rest eighth

  repeat 2 {
    play [C4, E4, G4] half velocity 90
  }
}
```

El lenguaje reconoce comentarios de línea y bloque, literales enteros, booleanos y strings, identificadores, operadores aritméticos, relacionales y lógicos, y las palabras reservadas del dominio musical.

Las expresiones se utilizan en declaraciones de variables, velocidades, cantidades de repetición y condiciones. El lenguaje soporta scopes anidados para variables dentro de bloques de repetición y condicionales.

La sintaxis musical soporta:

- Notas con pitch y octava, como C4, F#5 o Bb3.
- Notas naturales con n, como Fn4, cuando existe una declaración de key.
- Acordes, como [C4, E4, G4].
- Silencios con duración.
- Velocidad MIDI opcional entre 0 y 127.
- Repeticiones con repeat.
- Condicionales con if, then y else.

3. Implementacion

3.1 Frontend

El frontend está implementado con Flex y Bison. Flex reconoce los lexemas del lenguaje y Bison define la gramática, precedencias y acciones semánticas que construyen el AST.

La especificación léxica se encuentra en `src/main/c/frontend/lexical-analysis/FlexPatterns.l`. Dentro se definen las palabras reservadas musicales, los tipos, operadores, duraciones, comentarios, pitches, note classes, strings, enteros e identificadores.

La gramática principal está en `src/main/c/frontend/syntactic-analysis/BisonGrammar.y`. El no terminal inicial acepta una lista de configuraciones globales seguida por una lista de pistas. Las producciones construyen nodos para settings, tracks, eventos, acordes, declaraciones de variables y expresiones. Las acciones de construcción están separadas en `BisonActions.c`, lo que mantiene la gramática más declarativa.

El AST se modela en `AbstractSyntaxTree.h` y se libera en `AbstractSyntaxTree.c`. Esta separación permite que las etapas posteriores trabajen con una representación estructurada del programa sin depender de los detalles de Flex o Bison.

3.2 Backend

El backend está dividido en análisis semántico, generación de modelo musical y escritura MIDI.

El análisis semántico se implementa en `src/main/c/backend/domain-specific/SemanticAnalyzer.c`.

Sus responsabilidades principales son validar:

- Tempo positivo.
- Compás con numerador y denominador válidos.
- Tonalidad con nota base válida.
- Nombres de pista únicos.
- Instrumentos soportados: piano, guitar, bass, violin, flute y drums.
- Pitches dentro del rango de octavas 0 a 8.
- Uso de n solamente cuando hay una declaración de key.
- Velocidades MIDI entre 0 y 127.
- Tipos de variables y expresiones.
- Repeticiones con expresión entera.
- Condicionales con expresión booleana.
- División por cero cuando puede detectarse en constantes.

La generación de código se implementa en `src/main/c/backend/code-generation/Generator.c`.

Esta etapa baja el AST a un modelo musical intermedio, evalúa expresiones necesarias para velocity, repeat e if, asigna canales MIDI y prepara eventos temporales. Si no se especifican valores globales, usa tempo 120 y compás 4/4 por defecto.

La escritura del archivo MIDI se implementa en `src/main/c/backend/code-generation/MidiWriter.c`. El writer genera un Standard MIDI File de

tipo 1, con un track conductor para tempo y compás, y un track por cada pista del lenguaje. Para serializar correctamente los eventos utiliza delta times y valores VLQ.

El compilador escribe internamente `output.mid`, y el script de ejecución mueve ese archivo a `out/<nombre>.midi` para que cada ejemplo conserve su nombre.

3.3 Adicionales

El proyecto incluye ejecución con Docker Compose, scripts de build, run y test, y pruebas separada en casos de aceptación y rechazo.

También se agregaron ejemplos listos para ejecutar en la carpeta `examples/`:

- `twinkle-little-star.music`
- `bright-scale.music`
- `tetris.music`

El ejemplo de Tetris usa el tema tradicional Korobeiniki en estilo chiptune, con pistas separadas para melodía, bajo y percusión.

3.4 Dificultades Encontradas

Una dificultad central fue pasar de una sintaxis musical simple a un archivo MIDI válido. MIDI requiere distinguir eventos absolutos del modelo del lenguaje y delta times serializados. Además, los enteros de longitud variable obligan a codificar VLQ correctamente para que cualquier reproductor interprete el archivo.

Otra dificultad fue el manejo de canales. La percusión en MIDI no funciona como un instrumento melódico normal, el canal 9 tiene significado especial. Por eso el generador asigna drums a ese canal y saltea ese canal para instrumentos melódicos.

También requirió cuidado el manejo de scopes en variables. Las declaraciones dentro de bloques de repetición y condicionales deben convivir con variables externas, permitir shadowing en scopes hijos y rechazar duplicados dentro del mismo scope.

La funcionalidad de key quedó implementada parcialmente. El parser y el analizador semántico aceptan y validan key, y además se valida que el modificador natural `n` solo pueda usarse si existe una tonalidad. Sin embargo, la generación MIDI todavía no aplica automáticamente las alteraciones propias de la tonalidad.

4. Futuras Extensiones

La extensión prioritaria es completar la semántica musical de key: una nota sin accidental debería transformarse según la escala mayor o menor declarada, y una nota con `n`

debería cancelar esa alteración. Esto permitiría escribir melodías de forma más natural en tonalidades con sostenidos o bemoles.

También podrían agregarse más instrumentos, control de volumen por pista, paneo, cambios de tempo internos, duraciones punteadas, tresillos, ligaduras, variables aplicables a octava y duración, y una salida textual de diagnóstico más orientada al usuario final.

5. Conclusiones

El proyecto cumple el objetivo principal, compilar un DSL musical propio hacia un archivo MIDI estándar. La solución integra frontend, validación semántica y backend, con tests y ejemplos ejecutables.

El lenguaje permite expresar piezas simples con varias pistas, instrumentos, notas, acordes, silencios, repeticiones, variables y condicionales. El backend genera archivos reproducibles y separa la lógica musical de la serialización MIDI.

El principal punto pendiente declarado es la aplicación automática de la tonalidad en generación MIDI.

6. Referencias

- src/main/c/frontend/lexical-analysis/FlexPatterns.l
- src/main/c/frontend/syntactic-analysis/BisonGrammar.y
- src/main/c/frontend/syntactic-analysis/BisonActions.c
- src/main/c/frontend/syntactic-analysis/AbstractSyntaxTree.h
- src/main/c/backend/domain-specific/SemanticAnalyzer.c
- src/main/c/backend/code-generation/Generator.c
- src/main/c/backend/code-generation/MusicModel.c
- src/main/c/backend/code-generation/MidiWriter.c
- src/main/bash/build.sh
- src/main/bash/run.sh
- src/main/bash/test.sh
- examples/twinkle-little-star.music
- examples/bright-scale.music
- examples/tetris.music

7. Bibliografía

- GNU Flex Manual.
- GNU Bison Manual.
- MIDI Manufacturers Association, Standard MIDI Files specification.

